



University of Asia Pacific

Department of Computer Science and Engineering

CSE 450: VLSI Lab

LAB REPORT

Experiment Number:01

Experiment Title: Design and Simulation of a 1-Bit Full Adder Using VHDL in Xilinx ISE

Submitted by:

Name: Halima Akter Shorna

Student ID: 22201101

Section: AB –(B)

Submitted to:

Md. Rasheduzzaman

Lecturer

Department of Computer Science and Engineering

Date of Submission: 20-09-2026

1. Experiment Name

Design and Simulation of a 1-Bit Full Adder Using VHDL in Xilinx ISE

2. Objective

The objective of this experiment is to design and implement a 1-bit Full Adder using basic logic gates and verify its functionality through RTL-level simulation. The experiment focuses on understanding how three binary inputs A, B, and Cin are processed to generate the Sum and Carry-out (Cout) outputs. The simulated results are compared with the expected truth table to verify the correctness of the design.

The main objectives are:

- To design a 1-bit Full Adder using basic logic gates.
- To understand the operation of the three inputs: A, B, and Cin.
- To generate the Sum and Carry-out (Cout) outputs.
- To implement the Full Adder at the RTL level.
- To verify the design through simulation and waveform analysis.
- To compare the simulation results with the Full Adder truth table.

3. Apparatus / Hardware & Software Requirements

Software Requirements:

Xilinx Vivado – for designing and implementing the Full Adder using RTL/HDL.

Vivado Simulator (XSIM) – for simulating the design and analyzing the output waveforms.

VHDL – for coding the 1-bit Full Adder.

4.Theory Of Full Adder

A Full Adder is a combinational digital circuit used to perform the arithmetic addition of three binary bits. Unlike a Half Adder, which adds only two input bits, a Full Adder also considers

the carry generated from the previous lower-order bit. A 1-bit Full Adder has three inputs and two outputs. The inputs are A, B, and Cin, where A and B are the two binary bits to be added and Cin is the carry input from the previous stage. The two outputs are Sum and Cout. The Sum represents the result bit of the addition, while Cout represents the carry generated when the addition produces a value greater than 1.

The Full Adder can be implemented using XOR, AND, and OR gates. First, A and B are added using an XOR operation. The result is then XORed with Cin to produce the Sum. For the carry output, the carry generated by A and B is combined with the carry generated by Cin using AND and OR operations.

5. RTL-Schematic Diagrams

The 1-bit Full Adder was designed using VHDL and implemented in Xilinx Vivado. The RTL schematic represents the logical structure of the design and shows how the three input signals, A, B, and Cin, are processed to generate the two output signals, Sum and Cout.

The RTL design implements the Full Adder logic using XOR, AND, and OR operations. After synthesizing the VHDL code in Vivado, the corresponding RTL schematic and block representation were generated.

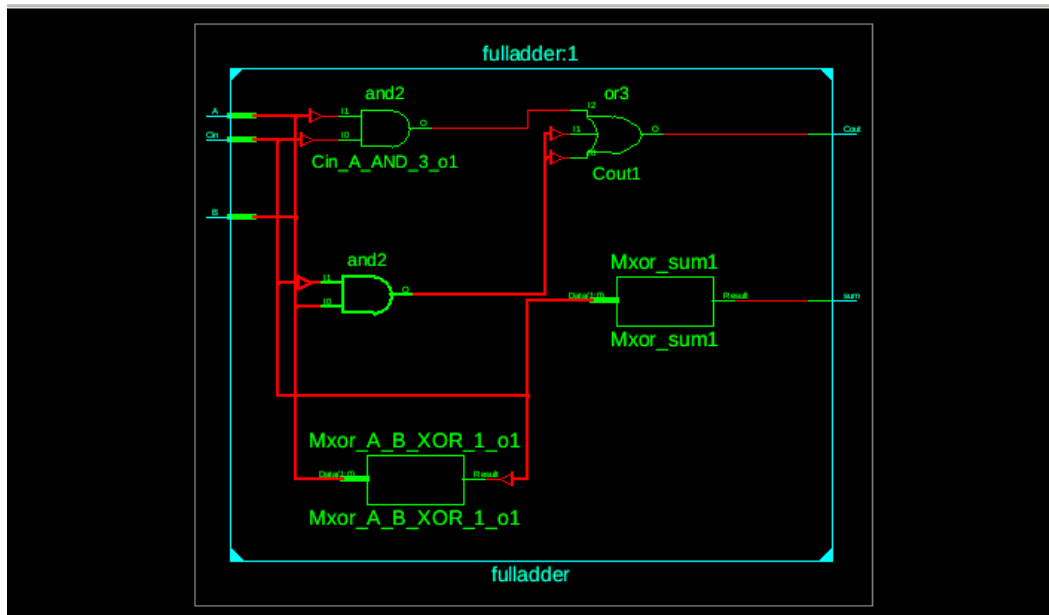


Figure 1: RTL Schematic of the 1-bit Full Adder

The schematic shows A, B, and Cin as the input signals and Sum and Cout as the output signals. The internal logic follows the Full Adder equations:

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

$$\text{Cout} = AB + \text{Cin}(A \oplus B)$$

The RTL block representation provides a simplified view of the Full Adder module. It clearly shows the three input ports (A, B, Cin) and two output ports (Sum, Cout). This representation is useful for understanding the overall input-output relationship of the design.

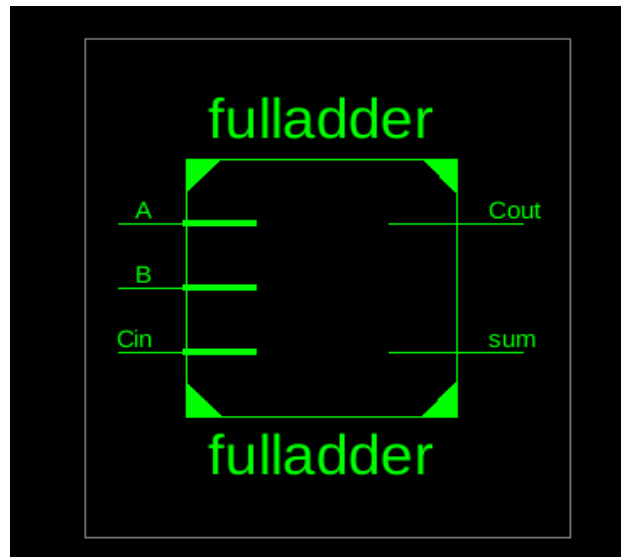


Figure 2: RTL Block Diagram of the 1-bit Full Adder

6.VHDL Code

The following VHDL code implements a 1-bit Full Adder. It takes three inputs, A, B, and Cin, and produces two outputs, Sum and Cout. The Sum is generated using XOR operations, while the Carry-out is generated using AND and OR operations.

Full Adder Design Code:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Full_Adder is
Port (
A : in STD_LOGIC;
```

```

B : in STD_LOGIC;

Cin : in STD_LOGIC;

Sum : out STD_LOGIC;

Cout : out STD_LOGIC

);

end Full_Adder;

architecture Behavioral of Full_Adder is

begin

Sum <= A XOR B XOR Cin;

Cout <= (A AND B) OR (B AND Cin) OR (A AND Cin);

end Behavioral;

```

The following testbench code is used to verify the functionality of the Full Adder. It applies all eight possible combinations of the three input signals. Each input combination is applied for 10 ns, and the corresponding Sum and Cout outputs are observed in the simulation waveform.

Full Adder Testbench Code:

```

LIBRARY ieee;

USE ieee.std_logic_1164.ALL;

ENTITY Full_Adder_tb IS

END Full_Adder_tb;

ARCHITECTURE behavior OF Full_Adder_tb IS

COMPONENT Full_Adder

PORT(

A : IN std_logic;

B : IN std_logic;

Cin : IN std_logic;

Sum : OUT std_logic;

Cout : OUT std_logic

```

```

);

END COMPONENT;

signal A   : std_logic := '0';
signal B   : std_logic := '0';
signal Cin : std_logic := '0';
signal Sum : std_logic;
signal Cout : std_logic;

BEGIN

uut: Full_Adder PORT MAP (

A   => A,
B   => B,
Cin => Cin,
Sum => Sum,
Cout => Cout
);

stim_proc: process
begin
A <= '0'; B <= '0'; Cin <= '0';

wait for 10 ns;

A <= '0'; B <= '0'; Cin <= '1';

wait for 10 ns;

A <= '0'; B <= '1'; Cin <= '0';

wait for 10 ns;

A <= '0'; B <= '1'; Cin <= '1';

wait for 10 ns;

A <= '1'; B <= '0'; Cin <= '0';

```

```

wait for 10 ns;

A <= '1'; B <= '0'; Cin <= '1';

wait for 10 ns;

A <= '1'; B <= '1'; Cin <= '0';

wait for 10 ns;

A <= '1'; B <= '1'; Cin <= '1';

wait for 10 ns;

wait;

end process;

END;

```

7. Output Analysis / Observations

After compiling and simulating the 1-bit Full Adder in Xilinx Vivado, the output waveforms of Sum and Cout were observed for all possible combinations of the input signals A, B, and Cin. Each input combination was applied for 10 ns in the testbench. The simulation results were found to be consistent with the expected Full Adder truth table. The Sum output becomes HIGH when the number of HIGH inputs is odd, while the Cout output becomes HIGH when at least two of the three inputs are HIGH.

Observation Table:

A	B	Cin	Sum	Cout
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

This table shows that the simulated Sum and Cout outputs match the expected results for all eight possible combinations of A, B, and Cin.

Simulation Waveform

The following waveform shows the changes in A, B, Cin, Sum, and Cout during simulation.

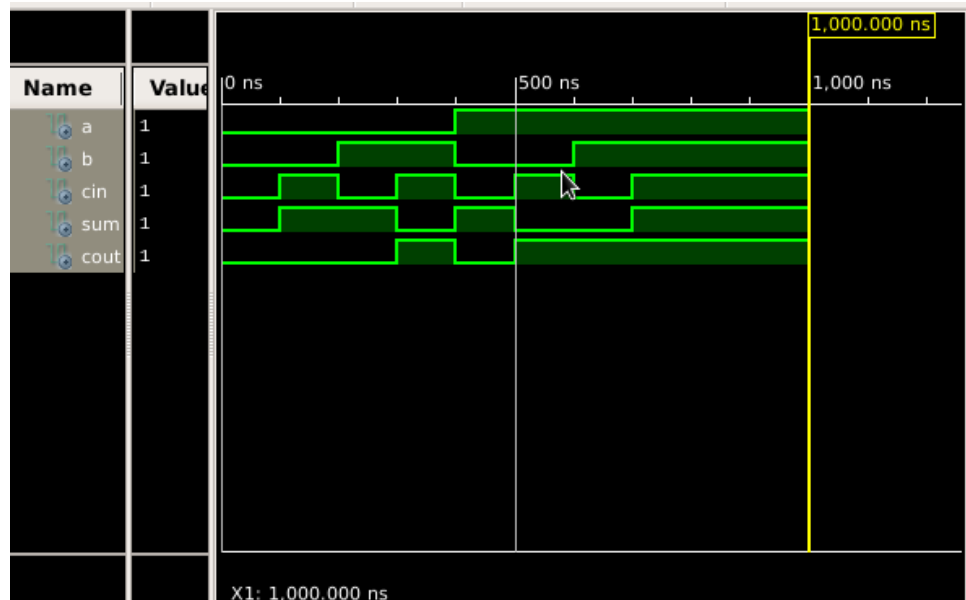


Figure 3: Simulation waveform of the 1-bit Full Adder

From the waveform, it can be observed that the Sum and Cout outputs change according to the applied input combinations. For example, when $A = 1$, $B = 1$, and $Cin = 1$, the output is $Sum = 1$ and $Cout = 1$,

Therefore, the simulation confirms that the designed 1-bit Full Adder operates correctly and produces the expected outputs for all eight possible input combinations.

8. Result

The 1-bit Full Adder was successfully designed and implemented using VHDL in Xilinx Vivado. The design was tested using a VHDL testbench by applying all eight possible combinations of the inputs A, B, and Cin. The obtained Sum and Cout outputs matched the expected truth table.

Therefore, the objectives of the experiment were successfully achieved, and the functionality of the Full Adder was verified through RTL simulation and waveform analysis.

9. Github Repository Commits

The Github Repository Link : <https://github.com/Shorna419630/VLSI-Project-Full-Adder-1-bit-.git>

10. Conclusion

Through this experiment, the working principle and design of a 1-bit Full Adder were studied and implemented using VHDL. The RTL design was created and simulated in Xilinx Vivado to observe the behavior of the Sum and Carry-out outputs.

The experiment helped in understanding how XOR, AND, and OR operations can be used to implement a Full Adder and how a VHDL testbench can be used to verify a digital circuit. The simulation results confirmed that the designed circuit produces the correct outputs for all possible input combinations.